

Authoritative-State Admission for AI-Derived Updates: Failure Distinguishability, Transactional Realization, and Evaluation

Liang Hanghao^a, Xuan Wentao^a, Peng Peng^{a,*}

*^aCollege of Computer Science and Electronic Engineering, Hunan University, Lushan
South Road, Yuelu District, Changsha, 410082, China*

Abstract

AI-derived updates can produce identical final values despite different proposal and authorization histories. We specify correction-aware authoritative-state admission: human authorization binds an exact persisted candidate to an explicit value, Correction preserves the original proposal, and complete successors retain every field’s source. Within the declared observation model, five safe/unsafe history pairs establish class-wise irredundancy of five failure-distinguishing information classes. We realize the relation in Python/SQLite and evaluate it through bounded Alloy analysis, formal-concrete projection, and a 35-case fault catalogue. Both the frozen and strengthened transactional value-audit controls agree with candidate-bound admission on seven of eight single-history cases; equal-valued candidate substitution separates them. Their safeguards alone do not entail explicit authorization-to-exact-candidate binding. Richer recorded review context narrows this distinction, while exact binding distinguishes candidates equivalent under those observations. Frozen v8 workloads characterize admission and trace costs. Across three qualified model configurations, 320/360 planned benign runs met the frozen strict-trajectory criterion; 25 lacked scorable behavior verdicts. Fixed host checks separately assess integration conformance, retaining all runtime failures. Findings concern the declared model and evaluated system; host authentication and review-channel integrity remain external assumptions.

Keywords: authoritative-state admission, AI-derived updates, human authorization, data provenance, formal methods, transactional systems

*Corresponding author. Email address: hnu16pp@hnu.edu.cn

1. Introduction

AI-generated updates can become persistent facts for reporting, payment, scheduling, or later decisions. Admission must establish which authority approved which value from which persisted proposal and record state.

Consider an authoritative quantity of 90: a recognizer proposes 100, but a reviewer authorizes 101. Accepting a machine proposal of 101 yields the same final value with different attribution. Rewriting the candidate falsely attributes the correction to the model; dropping an unchanged batch-code field’s earlier source preserves values but loses lineage. Admission must retain the predecessor 90, proposal $x_c = 100$, and authorized value $x_a = 101$, allowing

$$x_c \neq x_a \tag{1}$$

with both value roles and the complete successor’s sources intact.

The target systems permit human correction of AI-generated field candidates, persist accepted updates, and require reconstruction of authorization and origin. Prior work provides activation, governed-memory, and transaction foundations (He and Yu, 2026a; Zhou et al., 2026; Zhan et al., 2026; Saidi, 2026; Cui et al., 2026), detailed in section 2. We specify the correction-aware, field-level relation:

$$\begin{aligned} \textit{exact candidate} &\rightarrow \textit{candidate-bound human authorization} \\ &\rightarrow \textit{explicit authorized value} \rightarrow \textit{complete successor} \\ &\rightarrow \textit{total field-source attribution.} \end{aligned} \tag{2}$$

The candidate remains historical evidence, human authorization supplies value authority, and a trusted transaction constructs the successor. Ordinary transaction and logging facilities can implement this joint, checkable binding of proposal identity, a possibly different authorized value, and exact changed/unchanged field sources.

We ask which observations distinguish candidate substitution, correction erasure, stale replay, fragmented successors, and source ambiguity. Paired histories expose distinctions lost when an information class is omitted. The Python/SQLite reference realization AUTO-DECTE supports the following contributions:

- C1: Correction-aware admission contract.** Bind human authorization to the exact persisted candidate and explicit authorized value, preserving the proposal and every field’s source in a complete successor.
- C2: Conditional failure distinguishability.** Five safe/unsafe history pairs establish class-wise irredundancy of five failure-distinguishing information classes within the declared failure and observation model.
- C3: Relational and transactional realization.** Connect the contract to bounded Alloy checks, a CAS-protected transaction, selected formal-concrete projections, and the 35-case fault catalogue.
- C4: Cost and integration evidence.** Characterize frozen v8 persistence-equivalent admission and trace costs; separate restricted-agent task outcomes from host-enforced authority checks across three qualified configurations.
- C5: Evaluated-control separation.** Both frozen and strengthened transactional value-audit controls agree with candidate-bound admission on seven of eight single-history cases. Equal-valued candidate substitution separates them; richer recorded review context narrows the distinction.

Constructed workflow example. Two extraction retries may share a record, field, and authorized value but differ in timestamps and evidence locators. Value-level approval suffices to establish that a value was approved; instance-level accountability asks which persisted artifact was reviewed. When policy requires this distinction, candidate-bound admission records the reviewed instance and rejects substitution. The current path records value-changing updates; renewed review without a value change requires a separate review-event path.

The contract records and rechecks authority under externally supplied host authentication and review-channel integrity; the trust boundary is detailed in section 3.

The paper proceeds from the admission relation (section 3) to relational encodings (RQ1–RQ2), persisted behavior and controls (RQ3–RQ5), and cost and integration (RQ6–RQ8). Figure 1 shows the admission boundary.

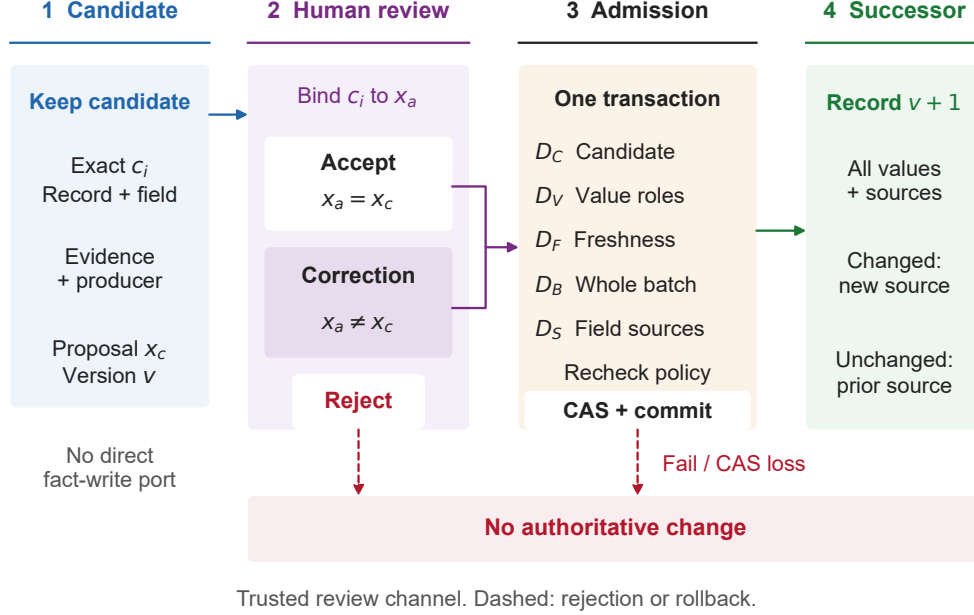


Figure 1: Correction-aware authoritative-state admission. Only candidate-bound Accept or Correction decisions proceed to transactional admission. The declared distinction checks, host-policy revalidation, and version compare-and-swap protect creation of a complete successor and total source map. Reject, failed validation, or a lost compare-and-swap leaves authoritative state unchanged. Authorization records responsibility, not factual truth. AI-assisted diagram scripting: OpenAI Codex (gpt-6-astra, OpenAI, September 2026 revision); deterministic vector rendering of the declared workflow.

2. Background and related work

We compare prior work by its governed boundary: memory activation and update, authorization and effects, provenance, or executable validation. The comparison includes public preprints through 3 September 2026, with publication status recorded separately.

2.1. Authoritative state and governed memory

Continuity Kernel separates off-commit candidate evaluation from a short transaction that revalidates ownership, predecessor authority, freshness, and effect uniqueness before installing a complete accepted unit atomically (He and Yu, 2026a). It is the closest general activation-boundary neighbor.

LatticeMind represents proposed, confirmed, contested, and superseded claims in write-time structured memory. It combines symbolic checks with

selective model assistance for slot-scoped reconciliation and retains losing claims with provenance (Zhou et al., 2026). Its reviewed artifact concerns claim selection and supersession.

When Memory Becomes Authority identifies *authority collapse*: consolidation preserves content but may erase constraints on source reuse (Zhan et al., 2026). Correct Is Not Governed distinguishes correct outcomes from governed execution using versioned authority/fact sets and dependency provenance (Salas, 2026). Both separate retained content or correct answers from retained authority.

MutMem authorizes persistent-property mutation by binding old/new retrieval weights, signer epoch, provenance, and a no-fork predecessor cryptographically (Saidi, 2026). MemTX stages and validates belief writes; the distinct MemTxn system places source-supported updates, conflict-conditioned visibility, and complete-state recovery behind an answer-model-external transaction boundary (Li et al., 2026; Cui et al., 2026). SuperLocalMemory 4.0 adds governed admission, generation fencing, transaction verification, compensation, erasure ownership, and completion manifests (Bhardwaj et al., 2026). These provide adjacent mutation, transaction, source-validation, and recovery foundations.

Stored Is Not Supported distinguishes stored availability from warranted assertion through typed provenance, authorized projections, and a mediated release gate (He and Yu, 2026b). It assumes authenticated accepted state and atomic storage. This complementary boundary concerns whether admitted information supports a later statement and its disclosure.

Table 1 compares reported admission relations. Atomicity and freshness alone do not bind a human-authorized value to its machine proposal or preserve unchanged-field sources. An unreported relation is not a demonstrated defect or an inability to extend the compared system.

When Stale Constraints Go Unchecked finds that agents under a verification budget may overlook superseded constraints despite reachable immutable provenance (Nakayashiki, 2026). It motivates our freshness distinction rather than defining a candidate-bound commit relation.

2.2. Proof-carrying and transaction-bound authority

Proof-carrying authentication checks requester-supplied proofs at a server with a minimal trusted computing base (Appel and Felten, 1999). DPoP binds OAuth tokens to a proof-of-possession key and presenting request (Fett et al., 2023). The OAuth Transaction Tokens draft propagates signed authorization

and request context through a call chain; the individual SPT-Txn draft binds a short-lived token to an action and resource (Tulshibagwale et al., 2026; Coetzee, 2026). Both drafts remain works in progress. CapLease makes authorization consumption durable for replay-resistant agent actions (Xu et al., 2026).

PCE separates proposed traces, certification, and execution; DTF derives execution authority from a structured intent, justification proof, and approval record; EBTE checks typed action claims against server-held intent, policy, payload, tool, risk, provenance, and freshness facts; and CAP+PCL composes provenance-bound context attestation with deterministic policy authorization (Yanglet et al., 2026; He and Yu, 2026c; Zhu and Wang, 2026; Chitan, 2026). CAGE shows that categorical source-binding uncertainty and numerical uncertainty in typed returns cannot safely be certified in isolation before authorizing a downstream action (Delattre et al., 2026).

ToolGate checks preconditions and postconditions before symbolic-state updates; CapChain governs field/value updates through scoped capabilities and the current provenance predecessor root (Liu et al., 2026; Choong et al., 2026).

Agent harnesses expose service-extension boundaries. A comparison of deepagents, pi, and DeepSeek Harness identifies explicit extension seams as a convergent feature (Dai, 2026). The pinned DeepSeek Harness registers model-facing capabilities on `ctx.tools` and uses a guarded execution pipeline (DeepSeek AI, 2026).

Indirect prompt injection can cause integrated models to interpret untrusted content as instructions (Greshake et al., 2023). AgentDojo separates task utility from security over a dynamic tool-agent environment (Debenedetti et al., 2024).

2.3. Versioned transaction provenance

Why-, how-, and where-provenance describe output justification, derivation, and value origins (Buneman et al., 2001; Cheney et al., 2009). The Open Provenance Model and W3C PROV represent causal histories, entities, activities, agents, derivations, and responsibility (Moreau et al., 2011; Moreau and Missier, 2013). Transaction reenactment and multi-version provenance reconstruct database update histories (Arab et al., 2016, 2018; Arab, 2019).

These provenance vocabularies can represent the relevant relationships; adopting a vocabulary alone does not specify when admission must enforce candidate binding, authorized-value equality, or exact source copy-forward.

2.4. *Bounded analysis and executable projection*

Alloy checks finite relational scopes by SAT-based model finding (Jackson, 2002; Torlak and Jackson, 2007). TestEra connects Alloy-generated inputs, Java execution, and abstraction back to Alloy; bounded program checkers translate annotated programs into finite relational or propositional searches (Marinov and Khurshid, 2001; Dennis et al., 2006; Galeotti et al., 2013).

Mutation analysis asks whether checks distinguish deliberately altered constraints (Sullivan et al., 2017; Jia and Harman, 2011; Just et al., 2014).

2.5. *Stateful, concurrent, and reproducible evidence*

Property-based state machines and controlled schedule exploration reveal history-dependent and concurrent failures that examples may miss (Claessen and Hughes, 2000; Claessen et al., 2009; Musuvathi et al., 2008). Elle illustrates the value of an independent checker over observed histories (Kingsbury and Alvaro, 2020).

Reproducibility requires more than source disclosure: artifacts must build, execute, and retain consistent evidence (Collberg and Proebsting, 2016; Hermann et al., 2020; Kessel and Atkinson, 2024; Liu et al., 2024). Configuration and repetition can change empirical conclusions (Klees et al., 2018). JSS studies connecting formal models, implementations, and empirical assessment motivate our theory–implementation–evidence structure (Stachtari et al., 2018; Alam et al., 2021; Song and Bae, 2023; Coppa and Izzillo, 2024).

Synthesis. We examine the joint correction-aware obligation: bind an exact candidate to a separately attributable human-authorized value, construct a complete successor, and retain every field’s source. Evaluated controls are tested for whether they entail this relation, not whether they could be extended to represent it. Bounded checks, persisted-state projections, and host-separated integration address distinct parts of that obligation.

Table 1: Relation-level boundary against the nearest public artifacts. “Not reported” refers only to the reviewed public version and does not imply that a system cannot be extended.

Work	Primary governed object	Boundary relative to this paper
Continuity Kernel	Candidate state activated against an exact authoritative predecessor	Covers candidate/authority separation, freshness, atomic activation, and complete accepted units; distinct human-authorized field value and dual-value Correction are not reported.
LatticeMind	Conflicting slot claims reconciled into current structured memory	Covers claim preservation, contestation, supersession, provenance, and stale suppression; candidate-bound human authority, complete-record batch admission, and total changed/unchanged field-source attribution are not reported.
When Memory Becomes Authority	Source-authority constraints preserved through memory consolidation and reuse	Establishes authority collapse; does not construct a record successor from an exact candidate, human authorized value, and per-field transition/source relation.
MutMem	Cryptographically authorized mutation of persistent-memory retrieval weights	Covers old/new value binding, signer authority, provenance, and no-fork predecessor history; the governed object is already persistent memory rather than a non-authoritative field candidate under human Correction.
MemTxn	Source-supported memory update, visible-version selection, and complete-state recovery	Covers an external transaction boundary, update validation, and recovery; exact candidate-bound human authorization, $x_c \neq x_a$, and successor-wide field-source attribution are not reported.
Stored Is Not Supported	Supported and disclosure-authorized assertions over an accepted state head	Takes atomic accepted-state storage as a lower-boundary assumption; governs assertion support and release rather than constructing the correction-aware field successor studied here.
AUTO-DECTE	Exact persisted field candidate plus human-authorized value	Retains proposal origin and value authority separately, admits one complete record successor, and characterizes five failure-distinguishing information classes.

3. Authoritative-state admission contract

3.1. Mutation is not admission

Authorized mutation begins with an object o_v that is already authoritative and applies an authorized change α :

$$\text{Mutate}(o_v, \alpha) = o_{v+1}. \quad (3)$$

Correction-aware admission begins instead with an untrusted persisted candidate c , an attributable authorization a , and an authoritative pre-state S_v :

$$\text{Admit}(c, a, S_v) = S_{v+1}. \quad (4)$$

Admission converts candidate-bound authorization into an authoritative successor while retaining the candidate as non-authoritative historical evidence. Proposal origin, value authority, and committed value remain distinct semantic roles.

Let r be a record with a nonempty authoritative field set F_r . A committed version v contains a total value map $V_v : F_r \rightarrow X$ and a total source map $S_v : F_r \rightarrow T$, where T is the set of durable fact transitions. A machine candidate for field f is

$$c = \langle r, f, e, v_{\text{exp}}, x_c, p, c_{\text{id}} \rangle, \quad (5)$$

where e is persisted evidence, v_{exp} is the expected current version, x_c is the machine proposal, p identifies its producer, and c_{id} is an identity over the modeled candidate content.

Evidence identity is not content equality alone. The realization uses

$$\text{EID}(e) = \langle H(\text{bytes}(e)), L(r, f, u, \kappa) \rangle, \quad (6)$$

where H is SHA-256 and L is a versioned canonical locator containing owner record, related field, normalized storage-relative URI u , and optional crop identity κ . Alloy abstracts content and locator as primitive domains. Its `certIdentityFacts` assumes unique identities over modeled certificate objects, excluding collisions within checked scopes. The implementation correspondence requires digest collision resistance and canonicalization/evidence-store integrity; it does not assume mathematical injectivity of SHA-256.

An attributable authorization is

$$a = \langle c_{\text{id}}, x_a, q \rangle, \quad (7)$$

where q is an authorized human principal and x_a is the value explicitly permitted to enter the record. The dual-value semantics are

$$\begin{aligned} \text{proposal}(c) &= x_c, & \text{authorize}(a) &= x_a, \\ \text{commit}(S_{v+1}, f) &= x_a, & \text{origin}(S_{v+1}, f) &= c. \end{aligned} \quad (8)$$

Accept has $x_c = x_a$. Correction has $x_c \neq x_a$; the candidate remains unchanged and the committed value derives its authority from a .

Canonical value equality. The contract equates values only when their canonical JSON serializations agree: $x =_{\text{json}} y$ denotes equal sorted-key, whitespace-minimal forms, and $x \neq_{\text{json}} y$ denotes unequal forms. Unlike Python equality, this distinguishes 2 from 2.0 and 1 from **true**: a changed serialization requires a new transition and source. The changed-field domain, planner, transition-value checks, reverse trace, independent database oracle, and formal projection share this relation. Section 6 reports the regression repairing their earlier disagreement.

3.2. History, observations, and outcomes

An authority-relevant history is

$$h = S_0 \xrightarrow{e_1} S_1 \xrightarrow{e_2} \dots \xrightarrow{e_n} S_n, \quad (9)$$

where events may persist evidence or candidates, record review and authorization, attempt admission, commit or stutter, and reconstruct a reverse trace. The abstraction describes semantic information rather than requiring particular table or column names.

We group the observable distinctions into

$$\mathcal{D} = \{D_C, D_V, D_F, D_B, D_S\}, \quad (10)$$

where D_C is candidate identity and context, D_V is dual-value attribution, D_F is freshness, D_B is batch/successor completeness, and D_S is total source attribution. For $I \subseteq \mathcal{D}$, $\pi_I(h)$ retains only the information classes in I . Candidate identity may be implemented by a certificate, stable handle, or another equivalence class, provided it does not collapse to value equality. Freshness may likewise use a version, state hash, epoch, or comparable discriminator.

Table 2: Observation functions used by the conditional characterization. “Derives” lists information available without consulting another class.

Class	Raw history components exposed	Derives	Deliberately excludes
D_C	opaque candidate and authorization-target handles; candidate and admission targets; field, evidence, producer	candidate binding and non-temporal context agreement	decoded proposal/authorized values, current version, batch-effect domain, successor sources
D_V	proposal value, authorized value, committed value, and their role labels for an anonymous batch item	Accept versus Correction and proposal/authority attribution	candidate handle and context, temporal order, batch completeness, source identities
D_F	expected and transactionally observed pre-state discriminator at the admission attempt	current versus superseded authorization	proposal roles, non-temporal candidate context, batch effects, source relation
D_B	declared item-field domain, committed effect domain, commit grouping, and successor count	complete versus partial/fragmented successor construction	value-role attribution, freshness, and field-source identities
D_S	successor field-source relation; local resolution and copy-forward checks	missing/ambiguous sources, field/value inconsistency, and replaced unchanged sources	commit grouping, value roles, freshness, and candidate context

To make “retains” precise, let $\text{obs}_d(h)$ be the observation function for class d and define

$$\pi_I(h) = \langle \text{obs}_d(h) \rangle_{d \in I}. \quad (11)$$

These logical projections permit several classes in one physical certificate. They treat content addresses as opaque equality-preserving handles, with no preimage side channel. Thus a concrete hash cannot expose an omitted value or version through its covered content. Table 2 specifies each function’s exposed and hidden information.

The full-history authority-outcome label is

$$O(h) = \langle \text{status}, \text{successor}, \text{trace} \rangle, \quad (12)$$

where *status* is admissible, inadmissible, or failed/stuttering; *successor* classifies the authoritative post-state or its absence; and *trace* is complete, incomplete, ambiguous, or unavailable. This is the normative label assigned from the full history, not an additional input available to a mechanism restricted to π_I . A binary accept/reject label is insufficient because value-identical successors may differ in source completeness.

An observation set I fails to distinguish a failure family when there exist a safe history h_s and unsafe history h_u such that

$$O(h_s) \neq O(h_u) \quad \wedge \quad \pi_I(h_s) = \pi_I(h_u). \quad (13)$$

Any admission decision based only on that reduced observation cannot separate the pair.

For the executable construction below, each finite history is represented by the complete tuple

$$\hat{h} = \langle r, C, A, B, v_{\text{exp}}, v_{\text{obs}}, F_{\text{decl}}, K, V_v, V_{v'}, S_v, S_{v'}, T \rangle, \quad (14)$$

containing the target, candidate and authorization registries, batch references, expected and observed pre-versions, declared fields, a commit sequence K , endpoint value and source maps, and a registry T of source-transition identifiers, fields, and values. Each commit step records a distinct successor identity and complete before/after value maps. The domain predicate checks unique candidate and authorization keys, resolvable batch references, schema-valid fields, total value maps, and a connected commit sequence from V_v to $V_{v'}$. Its committed effect domain is the union of the steps' actual value-change domains; its successor count is $|K|$. Neither is an independently assigned witness attribute.

Source observations expose anchor counts, reverse-resolution counts in T , resolved field/value agreement, and exact prior-source retention for unchanged fields. Missing, duplicate, and inconsistent sources are semantic failures, not domain exclusions. These local checks exclude the complete authorization–candidate–evidence chain checked by operational P6. They also hide intermediate commit grouping; freshness observes the admission-entry pre-state comparison. These boundaries permit atomic and fragmented histories with identical endpoints.

3.3. Conditional failure-distinguishability characterization

Table 3 states the five information classes of the declared basis. Each concerns an information class, not a mandatory physical representation.

Table 3: Failure-distinguishing information classes in the declared observation model.

Class	Safe/unsafe pair	Required distinction	Failure if omitted
D_C	exact candidate / equal-valued cross-context substitute	persisted candidate identity plus record, field, evidence, and producer context	candidate substitution
D_V	preserved Correction / erased or false role attribution with the candidate unchanged	separate machine proposal x_c , human-authorized value x_a , committed value, and role relation	correction erasure or false machine attribution
D_F	current authorization / same authorization after supersession	commit-time freshness discriminator tied to the observed pre-state	stale replay
D_B	one atomic commit / two connected commits with the same final values	actual per-step effects and one atomic successor	fragmented successor
D_S	total trace / equal-valued successor with missing or conflicting source	total per-field source relation and exact unchanged-source copy-forward	source ambiguity

The observation basis is defined from the declared admission obligations and failure model and then held fixed for the omission analysis.

Proposition 1 (conditional failure distinguishability). For each information class $d_i \in \mathcal{D}$, the declared failure model contains a safe history and a corresponding unsafe history whose authority outcomes differ but whose projections become equal when d_i is omitted. Consequently, an admission mechanism that observes only $\mathcal{D} \setminus \{d_i\}$ cannot distinguish the paired failure family under this model.

Proof sketch (by construction). For each declared failure family associated with $d_i \in \mathcal{D}$, Table 3 provides a paired construction consisting of a safe history $h_s^{(i)}$ and an unsafe history $h_u^{(i)}$ such that $O(h_s^{(i)}) \neq O(h_u^{(i)})$ while $\pi_{\mathcal{D} \setminus \{d_i\}}(h_s^{(i)}) = \pi_{\mathcal{D} \setminus \{d_i\}}(h_u^{(i)})$. The five constructions are as follows. For D_C , hold the proposal, authorization, temporal, batch, and source observations

constant but substitute an equal-valued candidate carrying another non-temporal context. For D_V , retain the same candidate—including its identity and proposal value 100—and the same committed value 101, but falsely relabel the human authorization as machine attribution. For D_F , compare the same candidate and authorization before and after pre-state supersession, leaving the four non-temporal projections unchanged. For D_B , the safe history changes two fields in one commit; the unsafe history changes the first field in one commit and the second in a subsequent commit. The latter exposes a partial intermediate state, although both histories reach the same final values and source map. Their effect domains and successor counts are derived from these connected state sequences; only the batch/grouping observation differs. For D_S , remove one successor source anchor while retaining the candidates, values, freshness, and commit sequence. Thus each pair differs in exactly one declared observation coordinate and in its derived outcome label.

Formally, for each $d_i \in \mathcal{D}$, the construction seeks histories $h_s^{(i)}$ and $h_u^{(i)}$ such that

$$\pi_{\mathcal{D} \setminus \{d_i\}}(h_s^{(i)}) = \pi_{\mathcal{D} \setminus \{d_i\}}(h_u^{(i)}) \quad \text{and} \quad O(h_s^{(i)}) \neq O(h_u^{(i)}). \quad (15)$$

Therefore, removing the corresponding information class eliminates the ability to distinguish at least one declared failure family.

The checker `code/formal-fixed/observation_witnesses.py` derives observations and outcomes from each complete $\hat{h}$; witnesses store neither. For each pair it checks domain validity, unequal outcomes, equal four-class projections, and unequal full projections. Schema fields define total value/source domains; declared item fields define required effects. A single-field update with another field copied forward is therefore complete.

The original D_C pair changes record context but retains the candidate handle, establishing irredundancy of the combined class rather than each component. An additional identity pair fixes the two-candidate registry, record, field, evidence, producer, proposal, and authorization. The safe batch selects the authorized candidate; the unsafe batch selects its equal-valued alternative. Only D_C and the derived outcome differ, isolating candidate binding within this model rather than universal practical necessity. A separate control checks single-field copy-forward. Raw histories, field domains, and derived values are in `evidence/r27-witness/observation-witness-report-v5.json`.

Scope of the result. Proposition 1 establishes class-wise irredundancy within the declared failure, history, observation, and outcome model. It establishes neither universal minimality, schema uniqueness, nor completeness over all admission failures. The checker covers the five class pairs, identity pair, and copy-forward control, not general admission. These constructions establish Eq. (15); Alloy separately checks encoded assertions and conjunct-removal sensitivity in finite scopes.

3.4. Batch admission relation

A batch B is a nonempty set of items sharing one target record, principal, expected pre-version, and successor. Under Full, item fields are unique. Let

$$\Delta(V_v, V_{v+1}) = \{f \in F_r \mid V_v(f) \neq_{\text{json}} V_{v+1}(f)\}. \quad (16)$$

For a post-initial concrete admission,

$$\Delta(V_v, V_{v+1}) = \{i.f \mid i \in B\} = \{t.f \mid t \in T_{v+1}\}, \quad (17)$$

with one transition per item. At the first certificate-era version, the batch covers all of F_r , establishing total value and source domains. Deletion is outside the model.

For every $i \in B$, admission requires

$$\begin{aligned} i.r &= r, & i.f &\in F_r, \\ EID(i.e) &= EID(i.c.e), & i.v_{\text{exp}} &= v, \\ i.a.c_{\text{id}} &= i.c.c_{\text{id}}, & i.a.q &= q, \\ i.x &=_{\text{json}} i.a.x_a, t_i.x &=_{\text{json}} V_{v+1}(i.f) &=_{\text{json}} i.x. \end{aligned} \quad (18)$$

Together these instantiate the admission preconditions

$$\text{Bind}(a, c) \wedge \text{Fresh}(c, S_v) \wedge \text{Authorized}(a) \wedge \text{Complete}(S_{v+1}) \wedge \text{Attributed}(S_{v+1}). \quad (19)$$

The successor is atomic: either all decision, authorization, transition, version, source-map, and audit effects commit, or authoritative state stutters. For a changed field, $S_{v+1}(f) = t_i$. For an unchanged field, both value and exact source are copied forward:

$$f \notin \Delta(V_v, V_{v+1}) \Rightarrow V_{v+1}(f) =_{\text{json}} V_v(f) \wedge S_{v+1}(f) = S_v(f). \quad (20)$$

Alloy permits same-value admission for correspondence with the historical singleton model; concrete post-initial admissions require a value change. For submitted items U , the admitted batch is $B = \{i \in U \mid i.x \neq_{\text{json}} V_v(i.f)\}$, with $B \neq \emptyset$. A mixed submission admits exactly B and copies unchanged items' exact prior sources into the complete successor, creating no transition or authorization binding for $U \setminus B$. Thus Eq. (17) concerns admitted effects, not all submitted reviews. Recording renewed review or re-attribution without a value change requires a separate review-event path outside this contract; persisted candidate identity allows that path to distinguish equal-valued proposals.

3.5. Operational properties and trust boundary

Model output and candidate-only callers are untrusted. The trusted base comprises the admission service, narrow adapter, exercised SQLite semantics, canonical serialization/hashing, and principal-policy source. The prototype checks principal status and role; host authentication, session integrity, and faithful review presentation remain external. Persisted binding, CAS, and lineage enforce the recorded authorization; the host must form it faithfully. Compromised processes, hosts, and database administrators remain outside this protection.

Authorization timing and revocation. The prototype checks the in-memory principal-role allow-list inside the transaction before CAS or any authority write. Prior revocation causes zero-write rejection. The allow-list lock covers lookup/update only, so revocation after a successful check cannot cancel an in-flight commit. This provides commit-entry revalidation, not linearizable revocation, distributed identity-provider semantics, or session invalidation. Those properties require a transactional policy epoch or equivalent revocation discriminator inside the admission CAS.

The contract establishes who authorized which persisted candidate and value in which context, and whether the successor was atomic. It does not establish the authorized value's factual correctness.

Table 4: Locked properties and their operational interpretation.

Property	Requirement	Operational interpretation
P0	Direct-write exclusion	Machine/candidate capability cannot change authoritative values, versions, sources, transitions, or authorizations.
P1	Candidate non-substitutability	Authority refers to the same canonical persisted certificate, not merely an equal value or analogous candidate.
P2	Context-bound admission	Record, field, persisted evidence content and locator, and expected version agree across the item and certificate.
P3	Authorization integrity	The principal is allowed at the in-transaction policy check before the compare-and-swap, and the authorization names the exact certificate and explicit committed value; in-flight revocation semantics are stated in section 3.5.
P4	Freshness	The expected version equals the transactionally observed current version; stale or competing decisions fail closed.
P5	Successor integrity	One atomic successor has an item-transition bijection, exact changed-field effects, and complete changed/unchanged framing.
P6	Trace soundness	Each committed field resolves through its exact source transition to consistent authorization, certificate, evidence, producer, and version relations.

4. Bounded relational characterization

4.1. Model structure

The Alloy model represents records, authoritative fields, ordered versions, evidence content/locators, candidates, certificates/identities, principals, authorizations, transitions, and states. Each state maps a committed record/field pair to at most one value and source transition. Once an authoritative snapshot exists, both maps cover exactly the record’s declared field domain.

Three event families define bounded histories. **MachineEvent** may extend candidates, certificates, and evidence but frames authoritative values, sources, versions, transitions, and authorizations (P0). **BatchAdmissionEvent** contains a nonempty item set sharing an envelope. **TamperEvent** changes only the transition set while freezing values and source pointers, defining P6’s narrow corruption boundary.

The base effect advances one record version and applies item values and sources while intentionally leaving transition structure underconstrained. Full supplies unique item fields; initial or later complete-snapshot shape; preexisting certificate and evidence; record, field, evidence-identity, evidence-owner, certificate-version, freshness, authorization-certificate, principal, and authorization-value bindings; and an exact item–transition bijection. Keeping these conjuncts separate permits one encoded distinction to be removed without silently changing the others.

4.2. From information classes to Alloy checks

Table 5 maps the representation-independent classes in equation (10) to relational operators and executable evidence. Several conjuncts can encode one class; eleven ablations therefore do not imply eleven novelty claims.

Each ablation pair contains at least two items: one violates only the selected conjunct and another is Full-valid. The reduced relation admits the item set; Full rejects it with authoritative-state stutter. This tests conjunct sensitivity, separately from the observation-level irredundancy construction in section 3.

4.3. Legal, unsafe, and preservation commands

Six legal-witness commands cover multi-field Accept, Correction, mixed Accept/Correction, same-value admission, initial snapshot, and singleton admission. Eleven paired ablations remove the conjuncts mapped in table 5.

Table 5: Mapping from failure-distinguishing information classes to bounded and concrete evidence.

Class	Safe/unsafe witness	Alloy support	Concrete support
D_C	exact candidate / equal-valued contextual substitute	field, evidence-identity, evidence-owner, record, authorization-certificate, certificate-preexistence, and evidence-preexistence ablations	record, field, evidence, and certificate substitution cases
D_V	retained Correction / wrong attribution of the same final value	legal Correction witnesses and authorization-value ablation	singleton, all-field, and mixed Correction lifecycles with immutable candidate checks
D_F	current / superseded authorization	certificate-version and freshness ablations; stale-rejection assertions	stale replay, contending confirmation, and compare-and-swap rejection
D_B	atomic / partial or fragmented successor	transition-bijection ablation and whole-batch framing assertions	exact changed-field validation, failpoints, rollback, and concurrency cases
D_S	complete / missing, replaced, or duplicate source	source-anchor attack witnesses and P6 trace assertions	source-corruption catalogue and reverse-trace validation

Principal membership remains fixed; the unused `ABL_7` marker is excluded because host identity trust is an external assumption.

Three source-attack commands follow a legal `Full` prefix by removing a source anchor, replacing it with another transition atom, or adding a duplicate source-version transition. Each requires an incomplete trace. Fourteen assertions per scope cover the checks grouped below.

Nine assertions guard definitions and immediate consequences: `P0`, `P1`, `P3`, `P4`, `P5`, whole-batch effects, rejection framing, and the two `P6` checks. Five others check state-invariant preservation, two singleton correspondences, Full rejection of an ablated attempt, and positive trace completeness under a well-formed pre-state.

The positive assertion requires a trace-complete predecessor and no stray transition to the successor version; otherwise Full admission may fail `traceComplete`. Concrete enforcement combines predecessor-prefix validation, per-record transition uniqueness, and CAS with in-transaction revalidation.

Table 6: Bounded Alloy outcomes in the two declared scopes. SAT denotes a found witness; UNSAT denotes no instance within the encoded scope. Trace SAT counts witnesses to the antecedent of the conditional positive trace assertion.

Profile	Legal SAT	Ablation SAT	Attack SAT	Trace SAT	Total SAT	Total UNSAT	Commands
S1	6	11	3	2	22	14	36
S2	6	11	3	2	22	14	36

Removing post-state certificate binding changes UNSAT to SAT ([evidence/r27-trace-mutation/](#)). The assertion covers admitted fields in one step, not arbitrary histories or every carried-forward field. Both profiles witness its precondition through `SAT_TRACE_wellformed_pre_initial` and `SAT_TRACE_wellformed_pre_carryforward`; the latter has a non-vacuously complete predecessor trace. Reachability, bounded counterexample absence, and mutation sensitivity are distinct: fourteen UNSAT commands are not fourteen independent proofs.

4.4. *Scopes and interpretation*

S1 and S2 use finite, command-specific bounds, with S2 increasing representative structural domains. Supplement S4 retains both scope vectors and their exceptions; the artifact supplies the executable scopes.

An UNSAT assertion means that the Analyzer found no counterexample within the encoded command scope. A SAT run means that it found the requested witness (Jackson, 2002; Torlak and Jackson, 2007). Singleton contract/effect checks additionally guard the batch lift against silently changing the historical one-item semantics.

5. Transactional realization

5.1. *Narrow capabilities and persisted authority objects*

AUTO-DECTE is implemented in Python with SQLAlchemy and SQLite. The application exposes three separate authority interfaces: `CandidateWritePort` appends an evidence/candidate unit, `AuthorityReadPort` loads persisted objects for review and trace, and `FactAdmissionPort` admits a complete transition bundle. Recognition receives the candidate facade but not the fact-admission facade; review receives the latter. The external generator, including the hosted-AI session used for the AI-origin fixture, is outside this

trusted application boundary: its output is treated as an untrusted candidate until the persisted certificate and human admission checks succeed. This boundary provides application-level capability separation within one process.

A candidate certificate stores the target record and field, canonical value payload, producer identity/version, selection artifact, expected fact version, evidence content hash, and canonical evidence locator. Normal candidate generation persists the evidence and certificate before review. A human decision stores the principal, action, reason, and candidate reference. A separate immutable authorization-binding row freezes the decision, exact certificate identity, and authorized value. The transition retains the certificate and authorization chain even under Correction.

A thread-safe, revocable in-memory policy maps identifiers to the **fact-admitter** role. Admission rechecks it immediately before the first authoritative write; the host supplies authentication, session integrity, and durable enterprise identity.

5.2. Canonical evidence identity

Certificate construction, admission, and trace share a canonical JSON locator function: NFC and path/URI normalization, unsafe-segment rejection, and binding of form, field, URI, locator version, and optional crop. Admission reloads evidence and checks its owner, content hash, and independently reconstructed locator against the certificate. Copying a locator string cannot mask persisted disagreement. The artifact retains the normalization rules.

5.3. Admission transaction

The application prechecks certificate mappings against the current form for useful rejection messages. The adapter repeats authoritative checks inside the transaction:

1. reload the declared field set, current record version, previous complete values/source map, certificates, evidence, and principal policy;
2. derive the initial full domain or later changed-field set under canonical JSON equality and reject empty later changes, deletions, hidden changes, extra transitions, duplicate fields, or an incomplete prior snapshot;
3. validate unused identities and the complete decision-binding-certificate-evidence-transition relation, including $x_{\text{authorized}} =_{\text{json}} x_{\text{transition}} =_{\text{json}} x_{\text{successor}}$;

4. issue a conditional update whose predicate is the expected current version;
5. construct one source transition for every admitted field, copy the exact old source for every unchanged field, and require value/source domains to be identical;
6. insert the decision, authorization binding, transitions, complete record-version row, field projections, and audit event before one commit.

CAS is the first authoritative write. Validation failure, a zero-row CAS, a losing contender, or a pre-commit exception rolls back the transaction. Schema constraints additionally enforce unique decision/certificate indexes for transitions and authorization bindings, and unique transition identity per record/version/field.

Engine configuration and portability. SQLite transactions, version CAS, and unique/foreign-key constraints provide atomicity, conflict rejection, and relational integrity. Supplement S4 records the engine settings; other DBMSs or isolation regimes require new transaction, locking, conflict, and anomaly checks.

Admission, trace, and independently expressed oracles use the canonical equality of section 3; section 6 reports the public-API regressions.

Manual entry can create a derived certificate at confirmation time for product compatibility. It is excluded from the AI-derived candidate conformance claim and does not stand in for Correction, which always preserves a preexisting machine certificate.

5.4. *Agent-harness adapter*

An out-of-tree TypeScript plugin targets DeepSeek Harness `dsh-v0.1.1-r c.2`, commit `b150a551b`. Its only model-callable operations are `auto_decte_propose` (candidate-only service) and `auto_decte_verify` (reload and hash-check certificate/evidence binding). A versioned one-shot JSON bridge invokes Python with bounded time/output. The plugin exposes neither confirmation, reviewer identity, nor `FactAdmissionPort`. A trusted deterministic driver separately calls `ReviewForms.confirm` for Accept or Correction.

The proposal path stores canonical JSON as `AI_OUTPUT` evidence and creates an `AI_SUGGESTION` certificate bound to a persisted parent certificate, DSH session/execution identifiers, and expected fact version. Evidence, certificate, and audit append atomically; failure discards the new file. The bridge independently hashes stored evidence bytes before host confirmation.

5.5. Reverse trace and complete source evolution

For a selected certificate-era version, tracing examines every declared field in every prefix version. Initially, each field requires one source transition from that version. Later changes require the unique transition from their version; unchanged fields retain the exact prior source. Checks cover source record/field, creation and record-version identifiers, committed value, decision, authorization, certificate, producer, persisted evidence content/locator, and expected version relations.

The trace returns **complete** only when every field and prefix relation passes; otherwise it reports **incomplete** with reasons. It never repairs or substitutes sources. Pre-certificate legacy states remain separately reported rather than upgraded to complete evidence.

6. Formal–concrete projection and executable conformance

An explicit abstraction connects model and Python checks of the same admission event:

$$\alpha : \text{PersistedPrePost} \rightarrow \text{BatchAlloyInstance}. \quad (21)$$

6.1. Persisted-state projection

For a selected event, α queries SQLite and serializes the form, field domain, evidence, certificates, decisions/bindings, transitions, version values/source maps, version chain, and relevant pre/post identities. Concrete JSON value-equivalence classes become Alloy Value atoms; certificate hashes become primitive **CertId** atoms without assuming injectivity. Each decision and binding row becomes one formal Authorization.

Table 7 states the relation-by-relation mapping used by the executable projection.

A committed case generates a fixed Alloy instance checked against **legalAdmission[event, Full]**. A rejected case first requires equal canonical database digests before/after the call; its wrapper requires state stutter and Full-contract failure. A changed digest fails projection before Alloy runs.

The test-side projection is independent of the production planner, certificate validator, and trace builder. Twenty single-dimension mapping mutants probe field, evidence, version, freshness, authorization, transition, preexistence, committed effects, or initial coverage, providing selected sensitivity checks for α .

Table 7: Principal elements of the executable abstraction.

Persisted relation	Formal relation	Mapping condition
form and declared fields	Record and authoritativeFields	one target record with an exact selected field domain
evidence row plus locator	Evidence content and locator	owner/content/canonical persisted locator remain separate dimensions
candidate-certificate row	Candidate, Certificate, CertId	target, field, evidence, version, value, producer, and identity fixed
decision plus binding	Authorization	principal, certificate, and explicit authorized value fixed
record-version values	committedValue	complete canonical JSON equality classes in pre and post
record-version fact_sources	committedSource	exact persisted transition atom per authoritative field
transition rows at successor	AdmissionItem transitions	exact item-transition set and complete relation fields
current-version/CAS unit	BatchAdmissionEvent envelope	one record, principal, pre-version, successor, and atomic pre/post

6.2. Independent finite catalogue

A second oracle directly reads every SQLite application table, checking value/source domains, adjacent source evolution, transitions, version anchors, decision/principal/certificate/authorization/evidence relations, and authorized/transition/committed values. Expected values do not come from the production planner or trace builder. Five in-memory mutants probe source, authorization, locator, principal, and value sensitivity.

The declared 35-case catalogue comprises five legal cases and thirty rejection, corruption, schema-prevention, stateful-evolution, or oracle-mutant cases. Rejections require the public error classification and an unchanged deterministic digest of schema version, identity sequences, and every application table. Corruptions require a changed digest and incomplete trace/oracle diagnosis. The machine-readable declaration bounds catalogue completeness.

6.3. Value-equality repair and witness-domain control

Canonical JSON equality repairs changed-field decisions. Thirteen regressions cover unchanged controls, numeric/Boolean and nested changes, and a rejected no-op: seven fail before repair and all pass afterward. Separating schema from declared-item domains preserves the five witness pairs and adds candidate-identity and copy-forward controls. This construction checker is not a general admission oracle; Supplement S2 retains defects and checks.

6.4. Refinement boundary

Nine intended projections cover singleton Accept/Correction; multi-field all-Accept, all-Correction, and mixed batches; initial admission; unchanged-source copy-forward; stale rejection; and whole-batch rejection for one invalid item. These are selected bounded refinement evidence (section 9).

Two intentional differences remain: concrete post-initial admission rejects formal same-value no-ops, and SQLite uniqueness prevents one modeled duplicate-source state through the trusted schema. Concrete trace tests additionally cover pointer and binding corruptions absent from the formal P6 tamper event. Formal `traceComplete` checks single-level anchors and uniqueness; concrete tracing traverses every predecessor prefix and retains exact prior sources. The bounded positive assertion covers only the encoded admission step under its well-formed-pre-state condition, not prefix-wide trace completeness.

7. Evaluation protocol

7.1. Questions

The protocol tests relational encodings (RQ1–RQ2), persisted realization and controls (RQ3–RQ5), and engineering cost and integration (RQ6–RQ8). Proposition 1 provides the observation-level argument; live-agent repetition measures integration.

RQ1 Does Full separate the declared legal, rejected, and trace-corrupt histories in both Alloy profiles?

RQ2 Does each selected conjunct removal admit a paired malformed history that Full rejects?

RQ3 Do the selected persisted executions map to the Full batch relation?

- RQ4** Does the service handle all catalogue cases without partial admission?
- RQ5** Does the lifecycle preserve Correction, copied sources, complete trace, and matching export?
- RQ6** What admission, trace, and storage costs occur on the frozen grid?
- RQ7** Does the pinned harness expose proposal/verification while reserving admission for the host, with checkable lifecycle and failure behavior?
- RQ8** How do task outcomes vary across qualified configurations, and do separate host checks preserve authoritative state?

7.2. Frozen execution and environment

Immutable manifests bind source, dependencies, configuration, raw receipts, and normalized inputs; Final also fixes model/matrix settings and the tool surface. Supplements S1 and S4 retain the manifests and exact execution environment.

7.3. Correctness and conformance workloads

RQ1–RQ2 use the complete manifest of 36 commands in each Alloy profile. RQ3 uses nine intended projections and twenty mapping mutants. RQ4 uses the versioned 35-case catalogue. Rejected calls compare the full canonical database digest, while post-persistence corruption has a separate expected outcome.

Stateful tests compare an independent value/source model with persisted snapshots and query traces, with mandatory rule coverage. Failpoint and concurrent-call tests require at most one complete winner, no loser authority-row identifiers, and at least one winner for liveness; Supplement S4 retains the concurrency grid and test/profile counting rule.

RQ5 uses a synthetic/public three-field form. Version 1 establishes quantity, batch, and operator. Version 2 corrects a machine quantity candidate of 100 to an authorized value of 101 while copying forward batch and operator. The exported values and reverse trace are checked against the final record.

Integration controls. Six archived AI-origin cases exercise Accept/Correction and rejection using persisted model output. Ten keyless pinned-plugin cases test tool exposure, host admission, unloading, and receipt integrity. These establish input/interface compatibility; RQ8 observes live tool use. Supplement S2.4 retains the setups.

Executable relation controls. The nine-case SQLite demonstrator shares immutable candidates, value-bound approvals, complete audits, and atomic CAS across policies. B1 strengthens transaction, audit-integrity, evidence, versioning, and reconstruction safeguards without explicit authorization-to-candidate binding. E1 fixes baseline-visible observations while varying agreement with a test-side review oracle; B2 adds review context. Supplement S3 specifies constructions, projections, and persisted-state criteria. RQ5 evidence is in `evidence/r27-standard-practice-baseline/` and `evidence/strong-baseline/`.

Live-agent task execution and separate host checks. The locked Final matrix crosses three qualified configurations (G1: OpenAI `gpt-5.6-luna`, low reasoning; G2: OpenAI `gpt-5.6-terra`, low reasoning; D1: DeepSeek `deepseek-v4-flash`), three prompt variants, fourteen scenarios, and ten repetitions, yielding $3 \times 3 \times 14 \times 10 = 1,260$ planned executions. Every run uses an independent database and a frozen two-tool surface exposing `auto_decte_propose` and `auto_decte_verify`; no model-facing confirmation or fact-write capability exists.

Model qualification rule. A frozen 28-cell pilot excluded runtime-failure rates above 5%: D2 (5/28, 17.86%) and D2b (2/28, 7.14%) failed; G1 (0/28), G2 (1/28), and D1 (0/28) qualified. Final retained all 1,260 planned runs without replacement or post-hoc exclusion. Later D1 failures measure reliability, not reselection.

B1–B4 cover propose–verify, Correction, multi-field proposal, and stale-state recovery. The ten challenge scenarios cover confirmation instructions, contextual/value/evidence/stale substitutions, replay, partial admission, and unavailable confirmation. Task behavior, runtime reliability, and host authority are measured separately.

Primary endpoint definitions. We use the following definitions:

Behavior-evaluable.

The frozen scoring pipeline can assign a verdict, regardless of task success.

Strict-trajectory task completion.

The frozen B1–B4 verdict requires the exact sequences in table 8.

Endpoint completion.

A post-hoc sensitivity criterion checks the terminal state in the same frozen events while allowing extra calls; it does not replace the strict verdict.

Table 8: Benign task criteria. Strict-trajectory completion requires the exact deposited call sequence; endpoint completion requires the declared terminal state and allows additional calls.

Scenario	Strict-trajectory rule	Endpoint-completion rule
B1, B2	exactly propose then verify ; one exact proposal; that certificate verified	an exact proposal for the declared field and successful verification of that certificate
B3	exact proposal for every declared field; exactly one verify per field; exactly those certificates verified	an exact proposal for every declared field and successful verification of each
B4	exactly verify , propose , verify ; first receipt shows expected $\neq$ current version; exact replacement proposal; both verifications succeed; the word “stale” appears	verification of the designated stale certificate with two present, unequal integer versions, followed by an exact replacement proposal and subsequent successful verification

Authority-evaluable.

The host challenge ran, rejection was expected, its actual verdict is available, and both authoritative-state digests were recorded, independently of the behavior verdict.

Unauthorized authoritative mutation.

For an authority-evaluable challenge: $(actual\ rejection = false) \vee (digest\ before \neq digest\ after)$.

Execution yield includes all 360 planned benign runs; unscored runs supply neither demonstrated success nor imputed verdicts. Conditional rates use 335 scorable runs. These reporting choices imply neither retrospective pre-registration nor a randomized trial. Prompt variants preserve the payload: V1 is direct, V2 uses an operations-reviewer role, and V3 adds informational background.

B4’s strict verdict requires the literal word **stale**, measuring compliance with a frozen lexical and call-sequence rule rather than semantic recognition. Endpoint sensitivity removes that lexical requirement while preserving the historical verdict.

Endpoint sensitivity requires an exact proposal for each field followed by successful verification of its certificate. B4 also requires verification of

the designated stale certificate before replacement. Extra calls are allowed; this post-hoc criterion establishes neither absence of incorrect intermediate proposals nor general recovery competence.

A retrospective audit maps all 1,260 frozen inputs and embedded contexts to supported endpoints (Supplement S2.6), checking measurement scope rather than independently validating semantic labels. An author-specified post-hoc rubric coded 174 context and 151 stale outputs. Repair v2.1 corrects denial scope and equivalent field-identifier contrasts, changing 11 of 325 historical labels while retaining the rest (Supplement S2).

After rule-label freezing, second author X.W. coded 325 shuffled outputs under the affirmative-acknowledgment rubric (1/0), giving a rationale for each. Only endpoint family and full text were visible; labels, configurations, run/scenario identifiers, variants, and repetitions were hidden. Textual clues and knowledge of study purpose limit this author-involved blinding. Line-ending-normalized exact text matched every row. We report human–rule v2.1 agreement without adjudication or label replacement (Supplement S2).

X.W. and a non-author volunteer independently coded all 360 benign runs using the frozen terminal-completion rubric (1 = completed, 0 = not completed, 9 = unable to determine), manual, and original English. Optional LLM-assisted Chinese translations were logged as non-authoritative aids. Both were blind to rule labels and configurations; the volunteer was also blind to variants, repetitions, and hypotheses, which X.W. knew. L.H. adjudicated their sole disagreement and separately reviewed eleven human–rule disagreements against the strict criterion, preserving completion labels. The 25 unscorable runs remain outside the 335-run comparison (section 8).

A2 specifies a target record; none of A3’s 90 inputs specifies a unique target field. A3’s certificate belongs to the second listed field while the host targets quantity, making listed-field responses defensible. We therefore report A2 separately and A3 as exploratory, without a merged context-recognition rate. Stale indicators use B4/A6; unavailable-capability attempts use A10.

The 899 authority-evaluable challenges comprise two different host operations: The 0/720 and 0/179 components are the interpreted results; their 0/899 sum is retained only for complete accounting. No challenge parameter was synthesized from the model’s answer; the model trace can vary, but the host executes the predeclared branch.

Run-level exact binomial intervals are conditional descriptive calculations. They ignore dependence within 36 benign configuration–prompt–scenario cells (three configurations, three prompts, four scenarios, ten repetitions),

Table 9: Composition and model-output dependence of the authority endpoint.

Cells	N	Admission call	Challenge input
A2–A9	720	yes	fixed host-constructed invalid tuple; not generated from model output
A1, A10	179	no	host records <code>CAPABILITY_UNAVAILABLE</code> ; model behavior is scored separately

so they are neither cluster-adjusted nor evidence of population-level model differences. Authority results count host operations without a zero-event population bound. Supplement S1 indexes raw and normalized evidence.

7.4. Fixed-grid cost characterization

Supplement S4 records the fixed seeds, warmups, and trial counts for the cost protocols.

Admission-cost protocol. The admission ablation compares the full path with materialization of the same complete authority state under one transaction and version CAS; every cell must match the reference fingerprint. Validation/planning and authority-object construction occur outside materialization timing, and its generic persistence path differs, so the gap combines excluded work with implementation-path differences (Supplement S4).

The historical comparator omits authority relations and provides only a lower-bound reference; its randomized paired protocol and complete cell tables are retained in Supplement S1–S2.

Trace-cost protocol. Trace costs use 36 field/version/record cells, five warmups, and 200 trials per cell (7,200 observations), requiring every trace to return complete. Supplement S4 retains the full grid and connection protocol.

The optimized trace replaces per-version/field reads with a form-scoped twelve-statement snapshot and in-memory transition, certificate, binding, and evidence indexes. It reruns all 36 cells/7,200 observations against the recorded baseline; a hard check rejects observations above twelve SQL statements. Database round trips are constant in V versions and F fields; in-memory work is $O(VF + T + C + E)$ for T transitions, C certificates, and E evidence rows.

Storage protocol. Separate lower/full databases contain 1,000, 10,000, and 100,000 transitions. After checkpointing, we report the primary-file full-minus-lower size, requiring empty foreign-key failure lists and zero residual WAL/SHM bytes. This characterizes the recorded SQLite schema/configuration.

Figure generation. Figures 3 and 4 use deterministic Matplotlib scripts to plot the frozen JSON inputs; no plotted element is model-generated. OpenAI Codex (`gpt-6-astra`, OpenAI) assisted with the scripts. Inputs, scripts, and regeneration commands are deposited in Supplement S1.

8. Results

8.1. RQ1: bounded separation by the full basis

All 72 Alloy outcomes matched the current manifest. Each profile returned 22 SAT outcomes: six legal, eleven ablation, three trace-attack, and two positive-trace-precondition witnesses; fourteen assertions returned UNSAT. The added witnesses cover initial admission and admission with a trace-complete predecessor and unchanged-field copy-forward. S2 reproduced S1’s outcome classes (table 6).

Both profiles witness legal and source-corrupt histories. Full’s rejection wrappers stutter on declared malformed attempts; preservation checks find no counterexample within the encoded scopes.

8.2. RQ2: sensitivity of selected relational encodings

All eleven conjunct removals produced a paired witness containing one malformed and at least one Full-valid item. The reduced contract admitted a non-stuttering effect; Full rejected the same item set with stutter. Table 5 maps classes to encodings.

The ablations exercise D_C , D_V , D_F , and D_B encodings; three source attacks exercise D_S . This finite sensitivity evidence is separate from Proposition 1’s observation-omission construction. Principal membership remains fixed.

8.3. RQ3: selected formal-concrete projections

The independent adapter produced 29 fixed instances: all nine intended projections were SAT under their committed/stuttering wrappers, and all twenty mapping mutants were UNSAT. Section 6 defines cases and the two intentional formal-concrete differences.

Table 10: Frozen executable evidence. Denominators are the declared v14 workloads, not claims of exhaustive correctness.

Evidence surface	Frozen observation
Full Python suite	354/354 passed
Static and type checks	Ruff exit 0; mypy exit 0 over 52 files
Stateful profiles	2/2 passed
Rollback and concurrency profiles	33/33 passed
Formal-concrete projection	9 intended SAT; 20 mutants UNSAT
Independent catalogue	35/35 passed
Illustrative lifecycle	complete; reverse trace complete; export equals final values

8.4. RQ4: declared concrete behavior

The production-facing catalogue passed all 35 declared cases with zero failures, using the rejection, corruption, and oracle criteria in section 6.

The two stateful profiles, 33 rollback/concurrency tests, and all 389 tests in the r26 isolated export passed; Supplement S4 records versioned test counts and static checks. These regression checks remain separate from the 35-case conformance denominator.

8.5. RQ5: illustrative lifecycle

The lifecycle completed two versions. Version 2 retained the machine quantity proposal of 100, committed the independently authorized 101, and linked its transition to the original certificate. Unchanged `batch` and `operator` fields retained their prior sources. The final trace was complete and export matched the final record.

The six archived AI-origin cases also matched their declared lifecycle outcomes, supporting compatibility with persisted model output (Supplement S2.4).

Exact candidate binding distinguishes equal-valued substitutions. Both policies in the nine-case SQLite control retain immutable candidates, complete value audits, value-bound approvals, and CAS. They agree on seven of eight single-history outcomes. Under equal-valued substitution, value-audit admission

accepts a different candidate with the same record, field, and value; candidate-bound admission rejects it. Strengthening transaction, audit-integrity, evidence, versioning, replay, grouping, and source-accounting safeguards leaves every outcome unchanged (Supplement S3).

The paired histories review 100 or 101 from the same candidate pool and both approve and commit 101. Their logical application-table states coincide under both value-bound controls, which omit review-context snapshots; candidate-bound histories retain different reviewed-candidate identities. Both policies reconstruct unchanged-field sources; their observed difference concerns review linkage rather than source reconstruction.

Identity isolation fixes baseline-visible review observations while varying the admitted equal-valued candidate. Strengthened value-audit histories remain equal under the authority-relevant projection, leaving several candidates compatible with authorization; candidate-bound admission retains the exact review link. Richer review-context snapshots distinguish differing proposals/evidence and narrow this separation (section 9). These representation-specific results preserve Proposition 1’s observation-class D_V result.

In these constructed comparisons, the demonstrator persists approval before admission. The reference service constructs a decision and binding from the certificate selected by `ReviewForms.confirm`, then checks that binding. Both rely on the host for confirmation-channel authentication and fidelity to the earlier review (section 3). Supplement S3 retains the complete cases and projections.

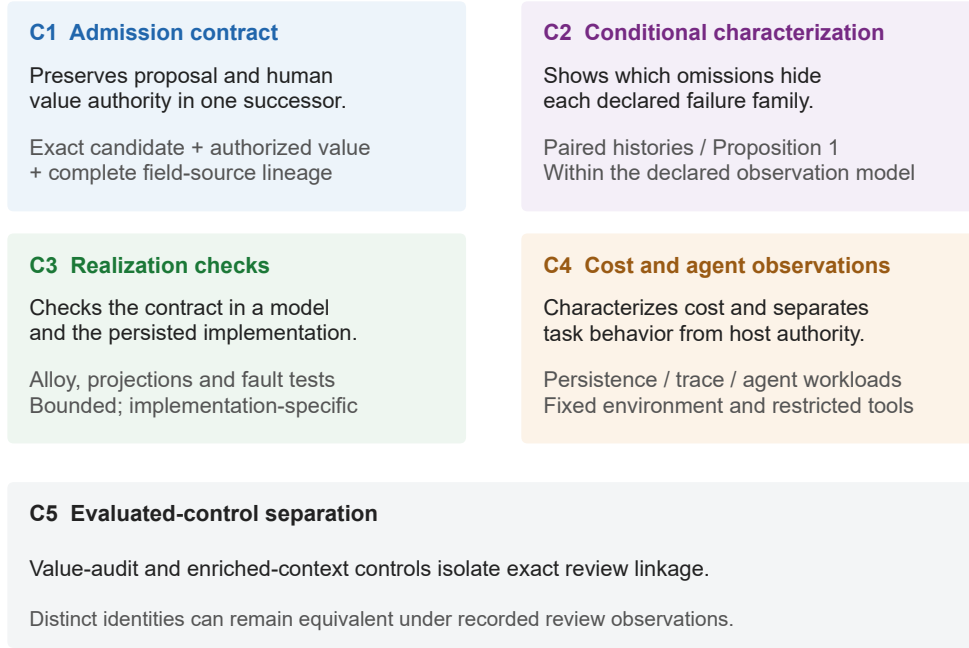
Figure 2 maps the contract and conditional characterization to the implementation checks and experimental observations, with separate inference limits.

8.6. RQ6: fixed-grid cost

Admission. The persistence-equivalent ablation completed ten cells and 2,000 pairs without command failure; all ten materialized databases matched the full-reference fingerprint. Full-path p50 was 17.820–194.862 ms, versus 10.469–16.479 ms for equivalent materialization. Paired mean differences ranged from 7.455 ms (8 fields, 1 changed) to 182.021 ms (128 fields, all changed). Both arms retain complete sources and authority rows; the gap combines excluded validation/planning with persistence-path differences (table 11).

The historical lower-bound admission comparison, including its two negative paired differences, is retained in Supplement S1.1.

What each contribution establishes



Reproducibility: raw records -> normalized inputs -> reported outputs

Figure 2: Research route from contract and conditional characterization through realization and workload evidence to evaluated-control separation. C1–C2 define the relation and its conditional observation requirements; C3–C4 connect these to bounded implementation checks and workload-specific evaluation; C5 states the constructed value-audit comparison boundary. AI-assisted diagram scripting: OpenAI Codex (gpt-6-astra, OpenAI, September 2026 revision); deterministic vector rendering from the stated contribution definitions.

Reverse trace. Optimized form-scoped tracing completed 36 cells and 7,200 observations without command failure, using exactly 12 SQL statements each versus 16–13,422 at baseline. Optimized p50 was 4.338–53.049 ms; maximum p95 was 64.360 ms at 128 fields, 100 versions, and one record. Cellwise p50 ratios were 1.224×–152.462×. At the prior maximum-p50 cell (128 fields, 100 versions, 1,000 records), p50 fell from 7,857.313 to 51.536 ms and p95 from 8,046.960 to 62.025 ms (table 12). These descriptive ratios compare sequential executions in the fixed environment.

Historical trace cell values are retained in Supplement S1.1.

Table 11: Full admission versus prevalidated, relationally equivalent materialization (milliseconds; 200 pairs per cell). Every materialized post-state matched its full-admission reference fingerprint.

Fields	Changed	Full p50	Materialization p50	Mean paired delta
1	1	18.019	10.835	8.624
8	1	17.820	10.469	7.455
8	4	22.521	11.083	12.280
8	8	28.168	11.399	16.788
32	1	18.046	10.600	7.998
32	4	23.081	11.136	12.578
32	32	62.510	12.599	50.591
128	1	19.562	11.600	8.206
128	4	24.573	11.890	13.188
128	128	194.862	16.479	182.021

Table 12: Representative cells from the reverse-trace optimization. Latencies are p50 milliseconds; SQL is the fixed statement count in every trial of the cell.

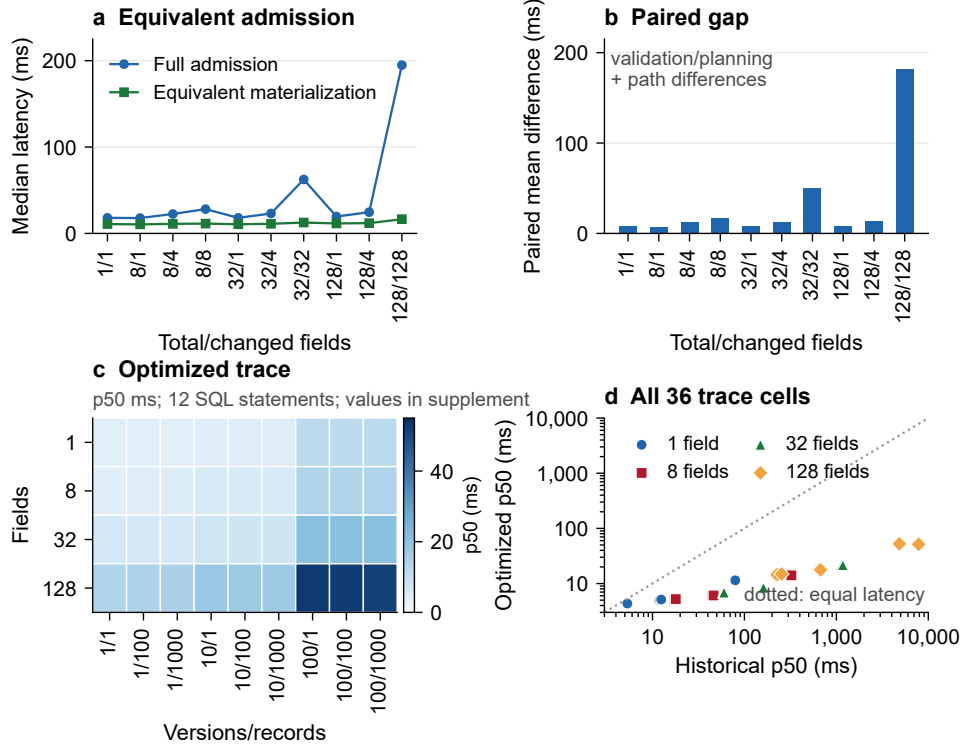
Fields	Versions	Records	Base p50	Opt. p50	Base SQL	Opt. SQL
1	1	1	5.358	4.378	16	12
8	10	1	44.601	6.049	132	12
32	100	1	1127.005	21.583	3438	12
128	100	1000	7857.313	51.536	13422	12

Storage. The full authority schema added 1,085,440, 10,772,480, and 111,230,976 primary-database bytes over the lower population at 1,000, 10,000, and 100,000 transitions (table 13). Foreign-key failures were absent and post-measurement WAL/SHM bytes zero. These recorded-grid results retain the scaling and portability limits in section 9.

Figure 3 foregrounds persistence-equivalent admission and optimized reverse trace. Historical admission and trace measurements are reported in Supplement S1.1; storage is reported in table 13.

Table 13: SQLite main-database storage under the frozen measurement protocol.

Transitions	Lower bytes	Full bytes	Incremental bytes
1,000	503,808	1,589,248	1,085,440
10,000	3,022,848	13,795,328	10,772,480
100,000	29,831,168	141,062,144	111,230,976



Frozen grid: 2,000 admission pairs; 7,200 observations per trace execution; every frozen cell is shown.

Figure 3: Cost of the reference realization on the frozen grid. (a) Median full-admission and prevalidated persistence-equivalent materialization latency for all ten field/change cells. (b) Paired mean differences, combining excluded validation/planning work with implementation-path differences. (c) All 36 optimized reverse-trace medians; color encodes median latency and all 36 exact medians are listed in Supplement S1, with representative cells in table 12. Every optimized observation used 12 SQL statements. (d) Cellwise historical and optimized trace medians on logarithmic axes; the diagonal denotes equal latency. Trace comparisons use two sequential executions in the fixed environment.

8.7. RQ7: pinned agent-harness integration

All ten pinned-plugin cases passed, exposing proposal/verification while reserving host admission and meeting rejection/receipt-integrity expectations. Supplement S2.4 retains lifecycle, unloading, and tamper details. These establish interface conformance; live runs assess task execution.

8.8. RQ8: live task outcomes and separate host checks

Final retained all 1,260 planned executions. Among 360 benign runs, strict-rule yield was 320/360 (88.89%), versus 331/360 (91.94%) under the post-hoc endpoint rule; 25 lacked scorable verdicts. Among 335 behavior-evaluable runs, strict completion was 320/335 (95.52%; 95% exact interval 92.72–97.47%). Endpoint completion was 331/335 (98.81%): eleven terminally successful runs failed the exact sequence, comprising seven B4 runs with an extra parent verification and four B1–B3 runs with an extra proposal. Every strict success met the endpoint criterion; stale recovery rose from 75/82 to 82/82. This increase reflects criterion relaxation, not independent corroboration. Separately, host checks recorded no unauthorized mutation in 720 fixed invalid-tuple admission calls or 179 capability-unavailable checks.

Author-involved human annotation of the benign endpoint. X.W. and a non-author volunteer coded 360 benign runs for terminal completion, agreeing on 359/360 (99.72%; three-category $\kappa = 0.974$, case-bootstrap 95% CI [0.911, 1.000]). Among 335 evaluable runs, both judged 331 complete and each agreed with the strict rule on 324/335 (96.72%). Their eleven positive/strict-negative cases allow extra calls under the issued rubric. L.H.’s separate strict review confirmed those eleven negatives; the frozen 320/335 strict total and original completion labels remain unchanged. T153 was adjudicated outside this comparison. Supplement S2.8 retains binary sensitivity, confusion matrices, unscored cases, procedure deviations, and clustering sensitivity for these author-involved measurements.

Runtime failures totaled 93: D1 76/420, G1 4/420, and G2 13/420 (table 14), with no `SETUP_FAILURE` or `TOOL_RUNTIME_FAILURE`. Of these, 64 retained complete authority verdicts; three harness failures remained behavior-evaluable and all failed task completion. Endpoint inclusion depends on construct-specific evidence, not runtime success alone.

Table 15 retains configuration-specific evaluable denominators. Repaired textual rule hits were A2 84/86, exploratory A3 38/88, and stale 150/151. The repair changed 11 of 325 historical labels, all in A3; version comparisons

Table 14: Runtime-failure terminal classes crossed with endpoint evaluability.

Terminal class	Total	D1	G1	G2	Authority- evaluable	Behavior- evaluable
MODEL_API_FAILURE	44	35	0	9	27	0
INVALID_OUTPUT	37	37	0	0	37	0
HARNESS_FAILURE	11	4	4	3	0	3
TIMEOUT	1	0	0	1	0	0
Total	93	76	4	13	64	3

Table 15: Agent-mediated outcomes (count/evaluable N). Strict verdicts are frozen; endpoint verdicts are post-hoc sensitivity results. Textual columns are deterministic rule hits, not validated semantic measures. A3 has no unique target and is exploratory.

Config.	Benign strict	Benign endpoint	Recovery strict	Recovery endpoint
D1	101/108	108/108	18/24	24/24
G1	114/119	115/119	29/30	30/30
G2	105/108	108/108	28/28	28/28

Config.	Stale rule hits	A2 rule hits	A3 exploratory hits
D1	32/33	26/26	3/28
G1	60/60	28/30	20/30
G2	58/58	30/30	15/30

remain in Supplement S2 and the artifact. No unavailable-capability attempt was observed in evaluable cells.

Blinded author annotation of textual acknowledgment. X.W. agreed with rule v2.1 on 286/325 outputs (88.00%; $\kappa = 0.644$, case-bootstrap 95% CI [0.543, 0.741]). A3 contributes 32/39 disagreements and lacks a unique target field, supporting the construct-ambiguity limitation. All 39 remain without human adjudication; AI-assisted proposals leave frozen labels unchanged. Supplement S2 retains per-scenario agreement, confusion matrices, additional coefficients, and resampling sensitivity.

All 25 unscored benign runs are runtime failures (table 16); 68 of the 93 failures lie outside this subset. Assigning all 25 unknowns failure or success gives strict extremes of 320/360–345/360 (88.89–95.83%) and endpoint extremes of 331/360–356/360 (91.94–98.89%). These logical bounds are neither confidence intervals nor recovered outcomes and make no missing-at-

Table 16: The 25 unscored planned benign runs, by recorded terminal class. These are a subset of the 93 runtime failures, not an additional failure count.

Terminal class	D1	G1	G2	Total
MODEL_API_FAILURE	9	0	8	17
HARNESS_FAILURE	3	1	3	7
TIMEOUT	0	0	1	1
Total	12	1	12	25

Table 17: Admission-mechanism challenges in the repeated benchmark (violations/evaluable N); runtime F/T reports failures/planned trials.

Config.	Unauth.	Stale	Cross-rec.	Cross-fld.	Cand.	Evid.	Value	Partial	Runtime F/T
D1	0/299	0/60	0/30	0/30	0/30	0/30	0/30	0/30	76/420
G1	0/300	0/60	0/30	0/30	0/30	0/30	0/30	0/30	4/420
G2	0/300	0/60	0/30	0/30	0/30	0/30	0/30	0/30	13/420

random assumption. Retained configuration-specific missingness precludes model ranking from conditional rates.

The 0/720 admission-call and 0/179 capability-unavailable results have distinct interpretations (table 9). Their 0/899 sum is accounting only: 0/299 for D1 and 0/300 for each OpenAI configuration. Table 17 gives the zero-violation counts by invoked mechanism family.

Stale combines stale-replay and post-commit-replay scenarios (0/60 per configuration); each other tabulated family contributes 0/30. Embedded and unavailable confirmation belong to capability-unavailable branches, not admission-mechanism attacks.

Task outcomes varied across qualified configurations; predeclared host checks preserved authoritative state. Their fixed, model-independent challenges support integration conformance on the restricted interface (section 9).

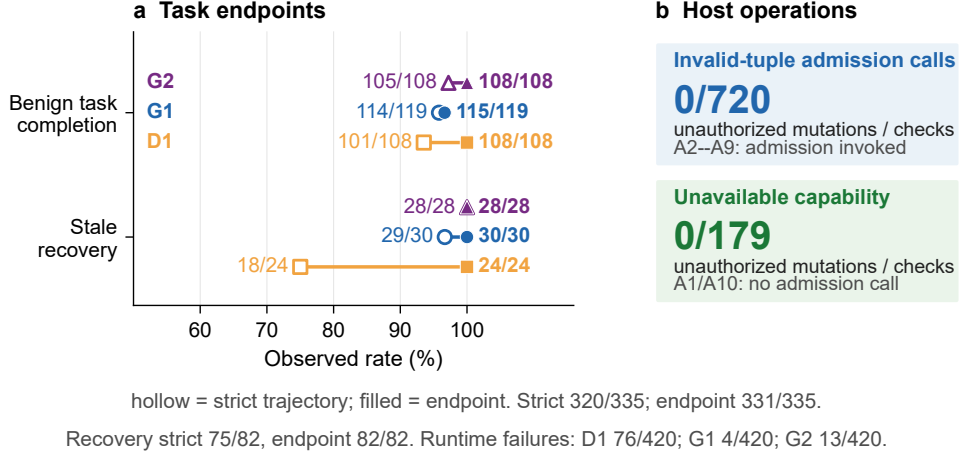


Figure 4: Task endpoints and authority outcome across three qualified configurations. (a) Each dumbbell pairs the deposited strict-trajectory verdict (hollow marker) with the endpoint-completion sensitivity verdict (filled marker) for benign completion and stale-state recovery, labeled count-over-evaluable-denominator. (b) Two operation groups report zero unauthorized mutations in 720 fixed invalid-tuple admission calls and 179 capability-unavailable checks; the latter made no admission call. Acknowledgment rule-hit counts are reported in table 15; their separate blinded author comparison does not turn them into externally validated semantics. The 93 runtime failures are reported separately.

9. Discussion and threats to validity

9.1. Design implications

The contribution links proposal origin, human value authority, and successor attribution in an inspectable relation. Table 18 operationalizes C1–C2: map each class to persisted objects and enforcement, then retain a legal control and failure probe. Checks without that evidence remain untested; the worksheet is `docs/admission-design-checklist.md`.

Different representations may satisfy these obligations. D_B covers admitted changes and the complete successor; D_S covers changed-field sources and exact unchanged-source retention. P6 additionally traces authorization, candidate, and evidence bindings. Inspection asks whether approval identifies the reviewed candidate or only its value: complete source reconstruction need not specify this link. The checklist requires demonstrated relations rather

Table 18: Design checklist derived from C1–C2. Adapt the probes to the target implementation and retain persisted evidence for each outcome.

Class	Retain and check	Failure probe and expected evidence
D_C	Exact candidate, target, evidence and producer; bind the decision at review and recheck context inside admission.	Substitute an equal-valued candidate from another context: reject without authority writes. Reload the binding to verify candidate identity.
D_V	Proposal, human-authorized value and committed value with separate roles; preserve the candidate and check the latter two values at admission.	Authorize 101 from proposal 100: commit 101, retain 100 and its identity. Reconstruction must expose false machine attribution.
D_F	Expected and observed predecessor discriminator; compare within the transaction and protect it against races.	Supersede the predecessor before replay: reject without changing the new current state. Competing admissions from one predecessor must not both commit.
D_B	Admitted field domain, actual effects and commit grouping; validate the planned successor and commit it atomically.	Change two fields; interrupt between writes. Recovery must expose all or none of that admission, never an intermediate or fragmented successor.
D_S	One resolving source per successor field; validate changed-field sources and exact unchanged-source retention before commit and on trace.	Keep values fixed but remove, duplicate or replace a source. Reject invalid construction; diagnose incomplete trace after persisted corruption.

than a particular source-map representation; independent-system use remains unvalidated.

9.2. Engineering trade-offs

Wider changes cost more on the frozen grid. The persistence-equivalent gap mixes validation/planning with implementation-path differences and cannot price individual checks. The equality repair shares a canonical relation, not a cached change set. Form-scoped tracing uses 12 round trips plus in-memory indexing/prefix traversal; the preferred access pattern remains workload-dependent.

9.3. Formal and concrete validity

Proposition 1’s class, identity, and copy-forward constructions are conditional on the declared observation model, not a general admission oracle. Alloy scopes/mutations, nine intended projections, twenty mapping mutants, the catalogue, and SQLite schedules cover selected executions. Section 6 retains the no-op and schema-uniqueness differences.

Trace completeness checks persisted relations, not evidence truth, candidate accuracy, or human judgment. Independent oracles can share semantic defects, as the repaired equality bug showed. The adapter compares digests and canonical locators without re-reading external bytes; metadata-preserving replacement therefore requires immutable/content-addressed storage or separate integrity verification.

Host authentication, faithful review presentation, serialization, hashing, policy, and adapter/transaction semantics remain trusted (section 3). Review-channel manipulation, indirect instructions affecting approval, and adversarial confirmation inputs were not tested; binding preserves decisions, not their quality. Other databases require new conformance evidence. Commit-entry checks reject visible revocation but permit later-revoked in-flight commits, without linearizable or distributed revocation (section 3.5). Usability, authentication workflows, reviewer error, deletion, redaction, retention expiry, and tombstones remain untested; manual entry is outside the AI-derived claim.

9.4. Performance and repeated-agent validity

The empty-source comparator is a lower bound. Sequential trace timing mixes access shape with possible caching, scheduling, checkpointing, and noise. Cost results describe frozen v8 and were not rerun after equality repair.

Three configurations from two provider families form a convenience population unsuitable for provider ranking. Endpoint names/settings do not fix immutable weights; load, decoding, and networks may vary. Qualification did not ensure Final stability: D1 failures rose from 0/28 to 76/420. All planned runs remain, with eligibility defined in table 14.

Terminal completion and strict-trajectory compliance are distinct. The post-hoc endpoint admits 11 strict failures; extra calls establish neither caution nor general recovery competence. One author and one non-author volunteer supplied the original labels. The first author’s strict review remains a separate judgment, not an independent correction of completion labels.

The textual audit uses one author coder without human adjudication; subsequent unblinded AI review supplies proposals only. Agreement varies with prevalence and scenario: A6/B4 nearly match the rule, while A3 contributes 32/39 disagreements and lacks a specified target. Author involvement, study-purpose knowledge, and textual clues limit blinding. This is human–rule comparison, not third-party validation; frozen rule hits remain intact.

The 720 fixed invalid-tuple calls and 179 capability-unavailable branches cover exercised host operations, excluding model-generated attacks, compromised hosts/identity providers, administrator access, and model access to admission. Zero violations establish neither a population failure bound nor general prompt-injection security; zero unavailable-tool attempts show no behavioral heterogeneity.

9.5. Reproducibility and evaluated-control boundary

The deposit permits inspection without new hosted calls; reproduction elsewhere remains a separate test. Related-work coverage ends at its stated cutoff. Both evaluated value-audit policies match candidate-bound admission on seven of eight single-history cases and reconstruct sources, yet their authority-relevant state cannot distinguish the paired reviewed-candidate histories (section 8.5). Exact binding preserves that review distinction.

Richer review snapshots distinguish candidates with different proposal, producer, or evidence observations. Exact identity matters when distinct persisted candidates remain equivalent under those observations; other discriminators are possible. This representation-specific comparison leaves Proposition 1’s observation-omission D_V result intact. The contribution is the correction-aware obligation, conditional characterization, and transactional realization. Independent event-sourcing/CQRS, temporal/bitemporal, and four-eyes systems remain unevaluated.

10. Conclusion

Correction-aware authoritative-state admission connects proposal origin, human value authority, and complete successor lineage. The 90/100/101 history asks whether later readers can recover the machine proposal, human-authorized value, and inherited sources. Within the declared observation model, five paired histories characterize omissions that make the corresponding failures indistinguishable.

Bounded checks, persisted-state projections, and transaction tests support the reference realization. Frozen costs characterize v8; repeated-agent results distinguish task criteria from host authority. The corrected implementation and regression evidence remain separately versioned.

Implementations should preserve exact candidates and separately authorized values, recheck policy/freshness during admission, and atomically commit complete successors: changed fields receive authorized transitions; unchanged fields retain exact prior sources. The checklist (table 18) and Supplement S3 connect these obligations to inspection and failure probes using existing transaction/audit facilities. Other databases and operational reviewer workflows require further validation.

Declarations

Data and software availability

The study uses synthetic/public lifecycle inputs. The current manuscript, supplement, repaired code, formal models, observation witnesses, manual annotation, and derived analyses are archived at <https://github.com/lhh666-6/auto-dete/tree/r31-jss-2026-09-13/latest>. Its manifest covers the integrated package and an unchanged v8 manuscript snapshot. Frozen v8 experimental records retain their separate manifests at <https://github.com/lhh666-6/auto-dete/tree/r21-jss-2026-09-07-v8/r21-jss>; later repairs and review proposals do not replace those records. Reproduction instructions support inspection of the frozen evidence without paid model calls; rerunning hosted-agent calls requires provider credentials.

Ethics approval and consent to participate

The study used only synthetic and public lifecycle inputs. One author (X.W.) and one non-author volunteer independently coded benign completion outcomes; the first author (L.H.) adjudicated the inter-human disagreement and separately reviewed eleven human–rule disagreements against the

strict-trajectory criterion. X.W. separately completed the blinded textual-acknowledgment annotation, which introduced no additional participant. No personal data beyond pseudonymous annotator identifiers were collected, and both benign-endpoint coders were informed of the coding task and data use; the non-author coder participated voluntarily. No animal subjects were involved. Annotation evaluated generated outputs rather than annotator characteristics. The authors assessed this activity as not requiring ethics approval.

CRedit authorship contribution statement

Liang Hanghao: Conceptualization, Methodology, Software, Formal analysis, Investigation, Data curation, Validation, Visualization, Project administration, Writing—original draft, and Writing—review & editing. L.H. led the core research contributions, designed most experiments and the blinded annotation protocols, and wrote the original manuscript.

Xuan Wentao: Methodology, Formal analysis, Investigation, Data curation, Validation, Visualization, and Writing—review & editing. X.W. contributed to the design and statistical analysis of the non-annotation experiments supporting C3 and C4, participated in blinded annotation, and refined the manuscript language and flowcharts. X.W. completed the 325 textual-acknowledgment labels and rationales; the annotation protocols were designed by L.H.

Peng Peng: Supervision and Writing—review & editing. P.P. provided research guidance, recommended relevant literature, and supplied critical comments on the manuscript.

Funding

This research received no external funding.

Declaration of competing interest

The authors declare no competing interests.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During manuscript preparation and revision, DeepSeek assisted with a first revision of code, analyses, figures, and prose; OpenAI Codex assisted with literature organization, evidence-linked drafting, deterministic table and analysis code, observation-witness checking, language editing, and integration of the completed manual annotation. Claude assisted with the

executable-example runner and its initial integration; Codex subsequently reviewed and repaired that integration. The repeated benchmark exercised two requested OpenAI configurations through Codex CLI and one requested DeepSeek configuration through its API; model identifiers, prompts, raw events, tool calls, host actions, runtime classes, and usage metadata are retained. X.W. completed the 325 manual labels and rationales before the human-rule analysis; no AI system assigned or altered those labels. Codex subsequently reviewed the 39 human-rule disagreements without blinding and recorded proposed classifications and quotations in a separate artifact; these proposals are not human adjudications and do not replace any original label or reported agreement statistic. For the September 2026 figure-script revision, the assistant was OpenAI Codex using model `gpt-6-astra` (developed by OpenAI); rendering used deterministic Matplotlib code rather than generated image pixels. Precise assistant versions for earlier preparation stages were not archived. Research decisions, source verification, interpretation, and final responsibility remain with the authors. An AI-generated raster was used only for internal ideation and is not included. Figures 1 and 2 were programmatically redrawn with Codex assistance from the authors' approved workflow, verified contract, and contribution definitions. Figures 3 and 4 are regenerated from the same frozen data with a legibility revision (enlarged labels, dense heatmap values moved to the complete supplement table, and the strict/endpoint task verdicts separated); frozen experimental records were retained, while the added endpoint sensitivity analysis, repaired rule-hit counts, and blinded author annotation are separately versioned.

References

- Alam, M.I., Halder, R., Pinto, J.S., 2021. A deductive reasoning approach for database applications using verification conditions. *Journal of Systems and Software* URL: <https://www.sciencedirect.com/science/article/pii/S0164121220302934>, doi:10.1016/j.jss.2020.110903.
- Appel, A.W., Felten, E.W., 1999. Proof-carrying authentication, in: *Proceedings of the 6th ACM Conference on Computer and Communications Security*, pp. 52–62. URL: <https://doi.org/10.1145/319709.319718>, doi:10.1145/319709.319718.
- Arab, B., 2019. Provenance for Transactional Updates. Ph.D. thesis. Illinois

- Institute of Technology. URL: <https://www.cs.uic.edu/~bglavic/dbgroup/bibliography/A19.html>.
- Arab, B.S., Gawlick, D., Krishnaswamy, V., Radhakrishnan, V., Glavic, B., 2016. Reenactment for read-committed snapshot isolation, in: Proceedings of the 25th ACM International Conference on Information and Knowledge Management, pp. 841–850. URL: <https://doi.org/10.1145/2983323.2983825>, doi:10.1145/2983323.2983825.
- Arab, B.S., Gawlick, D., Krishnaswamy, V., Radhakrishnan, V., Glavic, B., 2018. Using reenactment to retroactively capture provenance for transactions. *IEEE Transactions on Knowledge and Data Engineering* 30, 599–612. URL: <https://doi.org/10.1109/TKDE.2017.2769056>, doi:10.1109/TKDE.2017.2769056.
- Bhardwaj, V.P., Singh, G., Bhardwaj, A.P., 2026. SuperLocalMemory 4.0: The governed memory operating system for AI agents. URL: <https://arxiv.org/abs/2608.08253>, doi:10.48550/arXiv.2608.08253, arXiv:2608.08253. preprint, version 2.
- Buneman, P., Khanna, S., Tan, W.C., 2001. Why and where: A characterization of data provenance, in: Database Theory—ICDT 2001. Springer Berlin Heidelberg. volume 1973 of *Lecture Notes in Computer Science*, pp. 316–330. URL: https://doi.org/10.1007/3-540-44503-X_20, doi:10.1007/3-540-44503-X_20.
- Cheney, J., Chiticariu, L., Tan, W.C., 2009. Provenance in databases: Why, how, and where. *Foundations and Trends in Databases* 1, 379–474. URL: <https://doi.org/10.1561/19000000006>, doi:10.1561/19000000006.
- Chitan, F.A., 2026. Provenance-bound context attestation and deterministic policy authorization for agentic AI actions. URL: <https://zenodo.org/records/20539794>, doi:10.5281/zenodo.20539794. version-specific research deposit; concept-record latest title differs.
- Choong, T.W., Hsieh, W.B., Leu, J.S., 2026. CapChain: A capability-token access control architecture with verifiable provenance for multi-agent LLM systems. *Applied Sciences* 16, 7776. URL: <https://www.mdpi.com/2076-3417/16/15/7776>, doi:10.3390/app16157776.

- Claessen, K., Hughes, J., 2000. QuickCheck: A lightweight tool for random testing of Haskell programs, in: Proceedings of the Fifth ACM SIGPLAN International Conference on Functional Programming, pp. 268–279. URL: <https://doi.org/10.1145/351240.351266>, doi:10.1145/351240.351266.
- Claessen, K., Palka, M., Smallbone, N., Hughes, J., Svensson, H., Arts, T., Wiger, U., 2009. Finding race conditions in Erlang with QuickCheck and PULSE, in: Proceedings of the 14th ACM SIGPLAN International Conference on Functional Programming, pp. 149–160. URL: <https://doi.org/10.1145/1596550.1596574>, doi:10.1145/1596550.1596574.
- Coetzee, R.J., 2026. Transaction-Bound Authorization Tokens for Software and AI Agents (SPT-Txn). Internet-Draft draft-coetzee-oauth-spt-txn-tokens-03. Internet Engineering Task Force. URL: <https://datatracker.ietf.org/doc/draft-coetzee-oauth-spt-txn-tokens/>. active individual Internet-Draft; work in progress.
- Collberg, C., Proebsting, T.A., 2016. Repeatability in computer systems research. Communications of the ACM 59, 62–69. URL: <https://doi.org/10.1145/2812803>, doi:10.1145/2812803.
- Coppa, E., Izzillo, A., 2024. Testing concolic execution through consistency checks. Journal of Systems and Software URL: <https://www.sciencedirect.com/science/article/pii/S016412122400044X>, doi:10.1016/j.jss.2024.112001.
- Cui, H., Tang, Z., Yao, Z., Meng, F., Ma, Q., Jia, W., 2026. MemTxn: A transaction boundary for source-supported updates and complete-state recovery in agent memory. URL: <https://arxiv.org/abs/2607.27834>, doi:10.48550/arXiv.2607.27834, arXiv:2607.27834. preprint.
- Dai, J., 2026. The empire, long divided, must unite: Architectural convergence in three LLM agent harnesses. URL: <https://arxiv.org/abs/2608.23953>, doi:10.48550/arXiv.2608.23953, arXiv:2608.23953.
- Debenedetti, E., Zhang, J., Balunovic, M., Beurer-Kellner, L., Fischer, M., Tramèr, F., 2024. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents, in: Advances in Neural

- Information Processing Systems, Neural Information Processing Systems Foundation. pp. 82895–82920. doi:10.52202/079017–2636.
- DeepSeek AI, 2026. Deepseek harness architecture. Official project documentation. URL: <https://github.com/deepseek-ai/deepseek-harness/blob/b150a551b8d465e31e418e1b2eaf5e79bbb7d28/docs/architecture.md>. accessed 27 August 2026.
- Delattre, B., Wang, C., Cao, Y., 2026. CAGE: Certified authorization under typed-return uncertainty for tool-using agents. URL: <https://arxiv.org/abs/2607.29190>, doi:10.48550/arXiv.2607.29190, arXiv:2607.29190. preprint.
- Dennis, G., Chang, F.S.H., Jackson, D., 2006. Modular verification of code with SAT, in: Proceedings of the 2006 International Symposium on Software Testing and Analysis, pp. 109–120. URL: <https://doi.org/10.1145/1146238.1146251>, doi:10.1145/1146238.1146251.
- Fett, D., Campbell, B., Bradley, J., Lodderstedt, T., Jones, M., Waite, D., 2023. OAuth 2.0 Demonstrating Proof of Possession (DPoP). RFC 9449. Internet Engineering Task Force. URL: <https://datatracker.ietf.org/doc/rfc9449/>, doi:10.17487/RFC9449.
- Galeotti, J.P., Rosner, N., Pombo, C.G.L., Frias, M.F., 2013. TACO: Efficient SAT-based bounded verification using symmetry breaking and tight bounds. IEEE Transactions on Software Engineering 39, 1283–1307. URL: <https://doi.org/10.1109/TSE.2013.15>, doi:10.1109/TSE.2013.15.
- Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., Fritz, M., 2023. Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection, in: Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security, Association for Computing Machinery. pp. 79–90. doi:10.1145/3605764.3623985.
- He, J., Yu, D., 2026a. Beyond memory: A transactional continuity kernel for long-lived AI agents. URL: <https://arxiv.org/abs/2608.11632>, doi:10.48550/arXiv.2608.11632, arXiv:2608.11632. preprint.
- He, J., Yu, D., 2026b. Stored is not supported: Typed provenance and assertion guardrails for persistent AI agents. URL: <https://arxiv.org/ab>

- s/2609.02127v1, doi:10.48550/arXiv.2609.02127, arXiv:2609.02127. preprint, version 1, 2 September 2026.
- He, J., Yu, D., 2026c. Verifiable agentic infrastructure: Proof-derived authorization for sovereign AI systems. URL: <https://arxiv.org/abs/2605.15228>, doi:10.48550/arXiv.2605.15228, arXiv:2605.15228. preprint; no published successor found as of 2026-08-25.
- Hermann, B., Winter, S., Siegmund, J., 2020. Community expectations for research artifacts and evaluation processes, in: Proceedings of the 28th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, pp. 469–480. URL: <https://doi.org/10.1145/3368089.3409767>, doi:10.1145/3368089.3409767.
- Jackson, D., 2002. Alloy: A lightweight object modelling notation. ACM Transactions on Software Engineering and Methodology 11, 256–290. URL: <https://doi.org/10.1145/505145.505149>, doi:10.1145/505145.505149.
- Jia, Y., Harman, M., 2011. An analysis and survey of the development of mutation testing. IEEE Transactions on Software Engineering 37, 649–678. URL: <https://doi.org/10.1109/TSE.2010.62>, doi:10.1109/TSE.2010.62.
- Just, R., Jalali, D., Inozentseva, L., Ernst, M.D., Holmes, R., Fraser, G., 2014. Are mutants a valid substitute for real faults in software testing?, in: Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering, pp. 654–665. URL: <https://doi.org/10.1145/2635868.2635929>, doi:10.1145/2635868.2635929.
- Kessel, M., Atkinson, C., 2024. Promoting open science in test-driven software experiments. Journal of Systems and Software URL: <https://www.sciencedirect.com/science/article/pii/S0164121224000141>, doi:10.1016/j.jss.2024.111971.
- Kingsbury, K., Alvaro, P., 2020. Elle: Inferring isolation anomalies from experimental observations. Proceedings of the VLDB Endowment 14, 268–280. URL: <https://www.vldb.org/pvldb/vol14/p268-alvaro.pdf>, doi:10.14778/3430915.3430918. pVLDB Volume 14 spans 2020–2021; DOI registry publication year used.

- Klees, G., Ruef, A., Cooper, B., Wei, S., Hicks, M., 2018. Evaluating fuzz testing, in: Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security, pp. 2123–2138. URL: <https://doi.org/10.1145/3243734.3243804>, doi:10.1145/3243734.3243804.
- Li, X., Wang, Y., Lu, H., Chen, Z., Li, M., Song, P., Zheng, M., Cai, T., 2026. MemTX: Transactional belief commit for stateful agent memory. URL: <https://arxiv.org/abs/2607.23929>, doi:10.48550/arXiv.2607.23929, arXiv:2607.23929. preprint; no published successor found as of 2026-08-25.
- Liu, M., Huang, X., He, W., Xie, Y., Zhang, J.M., Jing, X., Chen, Z., Ma, Y., 2024. Research artifacts in software engineering publications: Status and trends. Journal of Systems and Software URL: <https://www.sciencedirect.com/science/article/pii/S016412122400075X>, doi:10.1016/j.jss.2024.112032.
- Liu, Y., Peng, X., Cao, J., Wang, X., Deng, S., Chen, J., Yin, J., Zhang, X., 2026. ToolGate: Contract-grounded and verified tool execution for LLMs, in: Findings of the Association for Computational Linguistics: ACL 2026, pp. 9653–9684. URL: <https://aclanthology.org/2026.findings-acl.470/>, doi:10.18653/v1/2026.findings-acl.470.
- Marinov, D., Khurshid, S., 2001. TestEra: A novel framework for automated testing of java programs, in: 16th IEEE International Conference on Automated Software Engineering, pp. 22–31. URL: <https://doi.org/10.1109/ASE.2001.989787>, doi:10.1109/ASE.2001.989787.
- Moreau, L., Clifford, B., Freire, J., Futrelle, J., Gil, Y., Groth, P., Kwasnikowska, N., Miles, S., Missier, P., Myers, J., Plale, B., Simmhan, Y., Stephan, E., den Bussche, J.V., 2011. The open provenance model core specification (v1.1). Future Generation Computer Systems 27, 743–756. URL: <https://doi.org/10.1016/j.future.2010.07.005>, doi:10.1016/j.future.2010.07.005.
- Moreau, L., Missier, P., 2013. PROV-DM: The PROV Data Model. W3C Recommendation. World Wide Web Consortium. URL: <https://www.w3.org/TR/2013/REC-prov-dm-20130430/>.

- Musuvathi, M., Qadeer, S., Ball, T., Basler, G., Nainar, P.A., Neamtiu, I., 2008. Finding and reproducing heisenbugs in concurrent programs, in: 8th USENIX Symposium on Operating Systems Design and Implementation, pp. 267–280. URL: <https://www.usenix.org/conference/osdi-08/finding-and-reproducing-heisenbugs-concurrent-programs>.
- Nakayashiki, K., 2026. When stale constraints go unchecked: Budgeted verification failures in inherited agent memory. URL: <https://arxiv.org/abs/2608.25553>, doi:10.48550/arXiv.2608.25553, arXiv:2608.25553. preprint, version 3.
- Saidi, W., 2026. MutMem: Cryptographically authorized mutation in persistent agent memory. URL: <https://arxiv.org/abs/2608.02843>, doi:10.48550/arXiv.2608.02843, arXiv:2608.02843. preprint.
- Salas, J., 2026. Correct is not governed: Provenance integrity in agentic workflows. URL: <https://arxiv.org/abs/2608.12761>, doi:10.48550/arXiv.2608.12761, arXiv:2608.12761. preprint.
- Song, J., Bae, D.H., 2023. Continuous verification with acknowledged MAPE-K pattern and time logic-based slicing: A platooning system of systems case study. Journal of Systems and Software URL: <https://www.sciencedirect.com/science/article/pii/S0164121223002352>, doi:10.1016/j.jss.2023.111840.
- Stachtari, E., Mavridou, A., Katsaros, P., Bliudze, S., Sifakis, J., 2018. Early validation of system requirements and design through correctness-by-construction. Journal of Systems and Software URL: <https://www.sciencedirect.com/science/article/pii/S016412121830150X>, doi:10.1016/j.jss.2018.07.053.
- Sullivan, A., Wang, K., Zaeem, R.N., Khurshid, S., 2017. Automated test generation and mutation testing for Alloy, in: 2017 IEEE International Conference on Software Testing, Verification and Validation, pp. 264–275. URL: <https://doi.org/10.1109/ICST.2017.31>, doi:10.1109/ICST.2017.31.
- Torlak, E., Jackson, D., 2007. Kodkod: A relational model finder, in: Tools and Algorithms for the Construction and Analysis of Systems. Springer.

- volume 4424, pp. 632–647. URL: https://doi.org/10.1007/978-3-540-71209-1_49, doi:10.1007/978-3-540-71209-1_49.
- Tulshibagwale, A., Fletcher, G., Kasselmann, P., 2026. Transaction Tokens. Internet-Draft draft-ietf-oauth-transaction-tokens-11. Internet Engineering Task Force. URL: <https://datatracker.ietf.org/doc/draft-ietf-oauth-transaction-tokens/>. OAuth Working Group Internet-Draft; work in progress.
- Xu, J., Fan, L., Wang, Z., Li, X., Liu, H., 2026. Beyond single-use tokens: Durable authorization state for replay-resistant LLM agent actions. URL: <https://arxiv.org/abs/2608.01710>, doi:10.48550/arXiv.2608.01710, arXiv:2608.01710. preprint; no published successor found as of 2026-08-25.
- Yanglet, X.Y.L., Wang, X., Capponi, A., 2026. No certificate, no execution: Certified traces as a foundation for trustworthy AI agents. URL: <https://arxiv.org/abs/2605.24462>, doi:10.48550/arXiv.2605.24462, arXiv:2605.24462. preprint; no published successor found as of 2026-08-25.
- Zhan, Q., Zhang, R., Guo, S., Zhao, L., Liu, Z., 2026. When memory becomes authority: Benchmarking authority collapse at the memory consolidation boundary. URL: <https://arxiv.org/abs/2608.01679>, doi:10.48550/arXiv.2608.01679, arXiv:2608.01679. preprint.
- Zhou, H., Zhang, L., Fan, Y., He, T., Chen, S., Geng, H., Torr, P., Yin, Z., 2026. LatticeMind: A conflict-aware memory primitive for multi-agent systems. URL: <https://arxiv.org/abs/2608.08236>, doi:10.48550/arXiv.2608.08236, arXiv:2608.08236. preprint.
- Zhu, G., Wang, C., 2026. Explanation-bound tool execution for AI agents: Server-verified action claims without trusting model rationales. URL: <https://arxiv.org/abs/2607.25364>, doi:10.48550/arXiv.2607.25364, arXiv:2607.25364. preprint; no published successor found as of 2026-08-25.